Internship report: Modularity in Interactive Theorem Provers

Arthur Adjedj
Université Paris-Saclay, ENS Paris-Saclay

Interactive theorem provers (ITPs) do not easily allow users to adapt and extend existing codebases without either changing the original code or duplicating it. The pattern of extending definitions occurs particularly often in programming language theory, type theory, and program verification, leading to a lot of code duplication and high maintainance burden. Existing approaches to this problem either require encoding datatypes as complicated expressions, obfuscating the development, or rely on meta-programming facilities which were underdeveloped in most ITPs until recently.
We develop **LeanExtend**: a tool implemented in Lean to produce modular code, leveraging the strengths of dependent types and modern meta-programming techniques

Note. *This research topic has been collabaratively developed by Arthur Adjedj and Yannick Forster. Arthur Adjedj was supervised by Yannick Forster and hosted in the Cambium team at the Centre Inria de Paris during a five-month internship.*

1. Introduction

Interactive Theorem Provers (ITPs), also known as proof assistants, are tools which allow for the development and mechanical verification of formal proofs. ITPs can be used to provide a very high level of guarantees for software (in particular the complete absence of bugs), and several projects make use of proof assistants to verify real-world software, such as the CompCert C compiler (Leroy [2009]), the sel4 operating systems kernel (Lyons et al. [2025]), and AWS' authorization policy language, Cedar (AWS [2026]). They have also been used successfully to formalise landmark theorems in mathematics such as the Four Colour Theorem or the Feit Thompson theorem, and the use of proof assistants is on the rise in mathematics departments, with big mathematical developments like Lean's Mathlib library (The mathlib Community [2020]) getting more and more public attention.

Like other programming languages, ITPs fall victim to the the fact that reusing definitions can be non-trivial. This is generally referred to as the **expression problem** (Wadler [1998]):

> "The goal is to define a datatype by cases, where one can add new cases to the datatype and new functions over the datatype, without recompiling existing code."

In fact, reusing and adapting existing data structures and programs modularly is a challenge for which few solutions have been produced over the years in industrial programming languages (Oliveira and Cook [2012]). It is even worse for ITPs, where in addition to programs, one needs to adapt proofs and dependently typed programs as well. Most projects that try to extend existing formalisation resort to copying the original content and adapting it for its own purpose. Since proofs about programs are arguably even harder to maintain than programs, this copy-paste approach incurs a huge maintenance burden.

"Meta-Theory à la Carte" (Delaware et al. [2013]) and "Pyrosome" (Jamner et al. [2025]) base their modularity on internal encodings of types (namely, impredicative church-encodings in the former, and Generalized Algebraic Theories in the latter). In both cases, the constructions are inneficient, incapable of extending previously user-defined inductive types (*i.e.,* expecting users to rely on the aforementionned encodings from the ground up), and expose the underlying internals of the encodings to the user. Having to deal with encodings of types rather than types adds a heavy burden in particular for new users interested in program verification. The lesson to include inductive datatypes natively

has been learned early by popular ITPs (namely Rocq and Lean), who at first had no primitive notions of inductive types in their systems and then resorted to add those. Nowadays, most systems usually posess a syntactic notion of (co-)inductive types, and justify the ability to define these via some classes of models, allowing users to work with the abstractions these types provide, rather than with their encoding in said models. The only popular system that still relies on encodings to define such types, and manages to hide their implementation details well, is Isabelle/HOL.

On the other hand, Rocq à la Carte (Forster and Stark [2020]) relies on the meta-programming capabilities offered by the MetaRocq Project (Sozeau et al. [2020]) to allow users to construct new inductive types and functions by merging other inductive types, and/or adding new constructors. New functions on a "merged" datatype can then be constructed by merging past functions. The metaprogram then uses the given piece of information to reconstruct a new inductive type, and new functions, based on the informations given by the user. While great for extending constructions "vertically" (*i.e.,* by adding constructors to a type), this system does not allow for horizontal extensions (*i.e.,* extending the type signature of inductive types and their constructors). Furthermore, this approach has been hindered in the past by the lack of good metaprogramming frameworks in ITPs.

None of these systems, independently of whether they use encodings or the meta-programming, handle all of the type-system of ITPs they are implemented for (*e.g.,* none handle co-inductive types), or allow users to extend past formalisations "after the fact". Instead, formalisations have to be built from the ground up with the expectation that they will be extended with a specific framework in mind, making them much less useful for real-world uses. AA: Mention Rocqet too.

We present **LeanExtend**, a new library, written in the Lean 4 proof assistant, which exposes new syntax for users to extend previous datatypes and declarations modularly, using meta-programming techniques. This in particular allows one to extend a previous formalisation that was not intended for such uses. To demonstrate this, we present two case-study,with one taking an existing, independent formalisation of the Simply-Typed Lambda Calculus (STLC), and extending it with new constructions, modularly proving the strong normalisation of the new calculuses, and another reimplementing Pyrosome's main case-study, which builds a formalisation of STLC and a translation pass to a call-passing-style (CPS) calculus, as well as extensions for both STLC and CPS, with extended translations between their respective extensions.

2. Inductive Types

2.1. What is an inductive type

With this framework constructed, we may now use it to construct practical partial mappings, by producing useful mapping contexts. Since most constants in proof assistants are either inductive constructions (i.e an inductive type, a constructor or a recursor), or a declaration (i.e a definition or theorem), we first focus on inductive extensions, and then look at how they can be used to construct useful declaration extensions.
First, let us look at the shape of an inductive type. Inductive types are shaped as follows:

$$\begin{aligned}&\textbf{inductive}\ \mathrm{Ind}\ \overrightarrow{(p : P)} : \overrightarrow{(\imath : I)} \to \mathtt{Type}\ \textbf{where}\\ &\mid \mathbf{ctor}_1 : \overrightarrow{(a_1 : A_1)} \to I\ \vec{p}\ \vec{d_1}\\ &\quad\cdots\\ &\mid \mathbf{ctor}_n : \overrightarrow{(a_n : A_n)} \to I\ \vec{p}\ \overrightarrow{d_n}\end{aligned}$$

An inductive (family of) type(s) is composed of a number of different components: A list of parameters $\overrightarrow{(p : P)}$ which are uniform in that they stay the same in every occurence of the type in its constructors, a telescope of indices $\overrightarrow{(\imath : I)}$ that may vary in each occurence, and a list of constructors

for this type. Each constructor contain a list of fields living over the context formed by the parameters, and instantiates the indices of the inductive type they inhabit in a context containing those fields. Basic examples of inductive types include the natural numbers, list, and vectors (i.e list indexed by their lengths):

```
inductive Nat : Type where
  | Z : Nat
  | S : Nat → Nat
```

```
inductive Vec (A : Type) : Nat → Type where
  | nil : Vec A Z
  | cons : (n : Nat) → A → Vec A n → Vec A (S n)
```

To each inductive type is associated a recursor, i.e a way to recursively eliminate a term of that type. For example, the recursor for `Nat` has the following type:
`Nat.rec` : $(\mathbf{P} : \texttt{Nat} \to \texttt{Type}) \to (\text{zero} : \mathbf{P}\ \texttt{Z}) \to (\text{succ} : (n : \texttt{Nat}) \to \mathbf{P}\ n \to \mathbf{P}\ (\texttt{S}\ n)) \to (t : \texttt{Nat}) \to \mathbf{P}\ t$
The existence of that recursor can be interpreted as the statement that, given a predicate $\mathbf{P}$ on natural numbers, an instance of that predicate on 0, and for every n, an instance of $\mathbf{P}\ (Sn)$ given $\mathbf{P}\ n$, that predicate holds for any natural number t. This corresponds exactly to the recursion principle for natural numbers.

2.2. Extending an inductive type

There are many apparent ways in which an inductive type may be modified/extended, i.e by either adding, removing or modifying either parameters, indices or constructors. We will only focus on adding constructors here. Consider an inductive A of parameters $\overrightarrow{(p : P)}$, indices $\overrightarrow{(i : I)}$ and constructors $\overrightarrow{\mathbf{ctor}}$, as well as another type B with the same parameters and indices, as well as the same constructors + a new constructor **newctor**, we can then map the type A to B and respectively every constructor of A to B's. The last missing piece to this translation is the recursor. A recursor has the shape A.rec : $\overrightarrow{(p : P)} \to \left(\mathbf{P} : \overrightarrow{(i : I)} \to A\ \vec{p}\ \vec{i} \to \texttt{Type}\right) \to \overrightarrow{(\mathbf{minor} : \ldots \to \mathbf{P}\ (\mathbf{ctor}\ \ldots))} \to \overrightarrow{(i : I)} \to (a : A\ \vec{p}\ \vec{i}) \to \mathbf{P}\ \vec{i}\ a$ where for every constructor, there is a `minor` As such, we can map this recursor to B.rec by simply mapping all the arguments straightforwardly, and leaving a metavariable hole for the minor of the new constructor:

$$\left[\!\left[\text{A.rec}\ \vec{p}\ \mathbf{P}\ \overrightarrow{\mathbf{minor}}\ \vec{i}\ a\right]\!\right] \coloneqq \text{B.rec}\ \overrightarrow{[\![p]\!]}\ [\![\mathbf{P}]\!]\ \overrightarrow{[\![\mathbf{minor}]\!]}\ ?m\ \overrightarrow{[\![i]\!]}\ [\![a]\!]$$

Our framework, provides a new Lean command which, given an inductive and new constructors, constructs another inductive type which extends the original one with these new constructors, adding the relevant mappings between from the first to the second:

```
inductive Var (α : Type) where
  | var : α → Var α

inductive Term α extends Var α where
  | lam : Term α → Term α
  | app : Term α → Term α → Term α
```

Additionally to the type, constructors and recursor that are primitively added by Lean, the system also generates lots of additional constructions that need to be mapped between the two types carefully, namely, given an inductive type `Foo`, Lean constructs the following list of auxiliary declarations, which each need get translated with particular care in **LeanExtend**:

- `Foo.recOn`: an alternative shape for the recursor
- `Foo.casesOn`: a recursor with no recursive hypotheses
- `Foo.CtorIdx`: a function mapping each recursor to a respective natural number
- `Foo.NoConfusion`: helper function for trivially proving inequality of terms with different constructors on each side

- `Foo.below`/`Foo.brecOn`: a recursor for "course-of-value" recursion, i.e recursion where the induction hypothesis applies to any strict subterm, a generalisation of the strong recursion principle for natural numbers. Useful for translating syntactically recursive functions into structurally recursive ones (see Section 3.1)
- `Foo.SizeOf`: helper function that maps each inductive term to a size, useful for proving the termination of a well-founded recursive function over that type (see Section 3.1)
- `Foo.ctor.inj`: injectivity lemma for each constructor

3. Definition extensions

Our framework now allows for inductive types to be extended into ones with additional constructors. we now look into how declarations (i.e definitions and theorems) can be partially mapped correctly. The initial idea is simple:

- take a given declaration
- partially map its term
- ask the user to fill-in the generated holes
- add the newly completed declaration to the environment
- map the new declaration to the old one

However, all of this turns out to quickly become complex, mainly because the way lean declarations are elaborated is in itself very complex. This can be clearly observed by looking at how much a user-written declaration differs from what it ends up being elaborated to. AA: TODO example that's ugly but not too much ?. Two big reasons why the elaboration is so complex are respectively how Lean handles recursion and pattern-matching. AA: Mention that the holes can be solved in the same way tactic holes are solved.

3.1. Recursive functions

Other proof-assistants like Rocq implement recursive functions using fixpoint constructs that are primitive to the abstract syntax, coupled with syntactic criterias for making sure recursive functions are well-founded. Lean, on the other hand, does not have such constructs. Instead, when a recursive function is defined, Lean tries to elaborate the function either into a structurally recursive term, using the recursor of whatever term decreases structurally in the recursive calls to write the function (à la McBride [1999]), or tries to prove the function be well-founded, morally recursing over the accessibility predicate.

Because of this, directly partially mapping the value of a recursive definition is not great, since it exposes internal encodings to the user, and asks one to manipulate those directly to extend the definition. Thankfully, Lean usually provides auxiliary lemmas that exhibit the original shape of the function that was defined. One of them, `foo.eq_def`, is a theorem of the form
`(a_1 -> A_1) -> ... -> (a_n -> A_n)  -> foo a_1 ... a_n = <foo's original definition>` As such, rather than use the original value of a definition, we can instead partial map the right-hand side of a function's `eq_def` whenever available, and then reuse the existing elaboration APIs Lean provides to translate such syntacticaly recursive functions to structurally recursive or well-founded functions. In particular, this allows us turn an originally non-recursive function into a recursive one if needed (e.g `Term.repr` example in the next section), or even change the termination measure that was originally used to adapt it to the new extended function.

3.2. Extending pattern-matches

Pattern-matching is a ubiquitous feature of modern functional languages, and proof assistants make no exception of that. In Rocq, matches are a primitive operation made explicit in the abstract syntax. In Agda, functions are defined as case trees, allowing users to branch on terms, similarly to regular

matches. Lean however does diverge from the tradition. For regular users, it may appear as though Lean does have match expressions: they are part of the concrete syntax and get appropriately pretty-printed when looking back at the abstract syntax too! However, this is all smoke and mirrors. Instead, Lean's elaborates pattern-matches down to the necessary inductive recursors, following technics described in Goguen et al. [2006]. Take for example a function of the form

```
def is_zero : Nat → Bool
  | 0 => true
  | n + 1 => false
```

The shape of the match get elaborated into an auxiliary function

```
is_zero.match_1 : (motive : Nat → Sort u_1) → (x : Nat) → (Unit → motive 0) → ((n :
Nat) → motive n.succ) → motive x
```

The function is then applied with the right arguments to explicit 1. the return type of the match (here called the `motive`) 2. the discriminant (here `x`) and 3. the righ-hand-side of each branch:

```
def is_zero : Nat → Bool :=
fun x => is_zero.match_1 (fun x => Bool) x (fun _ => true) (fun n => false)
```

In practice however, the pretty-printer recognises when an application has a match expression as its head, and thus prints it back to the user as a `match`.

When it comes to extending matches, we are now presented with two options:

- Matches are simply encoded using recursors, we may simply partially map a given matcher and ask users to fill in the holes in it.
- Extending matches amounts to adding new branches to it such that it covers the relevant constructors in the extended inductive type. As such, we may ask users to write down the relevant branches, in a syntax similar to how normal matches are written.

The second option turns out to be the most ergonomic, although it makes the implementation harder. In order to accomodate for this feature, when partially mapping the term of a given declaration, any application whose head is a matcher is translated into a metavariable hole, saving the original shape of the expression along the way. When trying to solve that metavariable, the original shape of the matcher is reconstructed, similarly to how the pretty-printer handles this expression. We then elaborate the new match branches provided by the user, and re-elaborate this into a new match-expression. Note that the implementation does **not** do any anti-quotations (i.e it does not construct concrete syntax from the original abstract syntax), and instead manipulates both the abstract syntax of the old matcher and the concrete syntax of the new branches in parallel, using Lean's existing internals for elaborating matchers. The end-result is a legible syntax for extending past declarations. Take the previous example of inductive extensions where `Term` extends `Var` in Section 2. Consider a function for printing a term of the type:

```
def Var.repr {α} [ToString α] : Var α → String
  | var x => s!"#{x}"
```

We may now extend this function for `Term` as such:

```
mod def Term.repr extends Var.repr where
  extend match_1 with
  | app f x => s!"{Term.repr f} {Term.repr x}"
  | lam f => s!"λ {Term.repr f}"
```

In practice having matches be handled specially provides a clear distinction between the holes generated for matches and those generated by explicit uses of recursors, which usually only appear in proof terms generated by tactic calls such as `induction` and `cases`. This in essence means our syntax for

modular definitions offers a clear separation of concerns between proof holes, solved by tactics, and non-proof holes, solved by completing match branches.

```
mod def <new function name> extends <old function name> where
  <match extensions>
finally
  <tactics>
<termination-information>
```

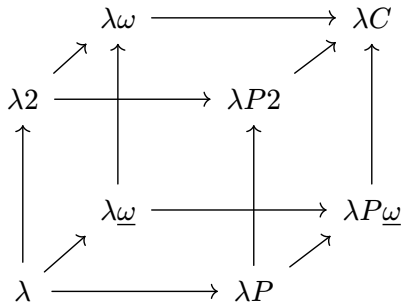
This sort of distinction is not new. Indeed, `where finally` is already a feature for definitions in Lean, and can be used to fill-in proof holes after the fact:

```
def Vec.tl (v : Vec α (S n)) : Vec α n :=
  match h : S n, v with
    | Z, nil => nomatch h
    | S k, cons _ _ tl => (show n = k from ?_) ▸ tl
                          --rewrites the type of `tl` from `Vec α k` to `Vec α n`
  where
    finally
    injection h
```

Similarly, Rocq's `Equations` (Sozeau and Mangin [2019]) syntax provide an "Obligations" system which serve a similar purpose. We thus adopt a similar syntax to normal Lean `def`, by asking for matchers to be completed first, before asking proof holes to be solved in a `where finally` block. Termination information such as `termination_by` or `decreasing_by` can be additionally provided if needed to help with producing a well-founded recursive function. This in effect means a user can adapt the termination measure of an extended definition, relative to the original's, a feature no other extension system provides to our knowledge. Said feature is of particular importance when extending a non-recursive type to a recursive one, as is the case, for example, with the extension from `Var.repr` to `Term.repr`

4. Making extensions modular

The current set-up allows one to extend previous definitions iteratively, though one may argue these extensions are not strictly "modular". In particular, one cannot simply "apply" an extension to adeclaration, and instead needs to ground his extensions on base declarations. This, in particular, means one isn't able to compose different extensions. Take the Barendregt lambda-cube (Barendregt [1991]) for exemple:



The various vertices of the cube can be seen as various extensions of the initial vertex corresponding to STLC. However, an important feature of this cube is that all vertices can be written as compositions of the 3 adjactent vertices of λ, namely λP, $\lambda 2$ and $\lambda\underline{\omega}$. If one wanted to formalise each vertex of the cube in the past, they would need to write down 8 different formalisations. With our current framework, they need only write down 1 formalisation and 7 extensions of that base formalisation. If our system was modular however, one would only need to write down the base formalisation

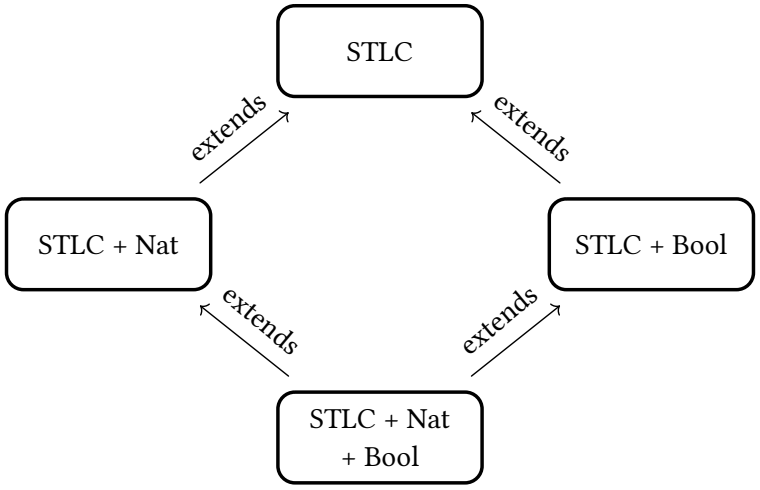
and the 3 adjacent extensions, only needing to compose them to get the rest of them afterwards. We focus back on our extension systems for inductive types and definitions and describe a way to make them composable, thus achieving true modularity.

4.1. Inductive modularity

4.2. Definition modularity

AA: Empty for now, no work has been done here, write down your ideas

5. Case study: extending STLC



In order to iterate on this implementation and ensure its usability, we studied the case of taking an existing implementation of STLC[1] which prove the strong normalization property of the system, and extended it with additional constructors, allowing us to modularly prove SN for the extended system. In particular, the original normalization was **not** built with modularity in mind, and was still extendable after the fact. The original formalisation is extrinsically typed, and follows the usual proof of normalisation using a logical relation.

```
inductive Ty : Type where
| base : Ty
| arrow : Ty -> Ty -> Ty
```

```
inductive Term where
| var : Nat -> Term
| app : Term -> Term -> Term
| lam : Ty -> Term -> Term
```

We extend this base definition to add natural numbers:

```
inductive Ty extends Ty where
  | nat
```

```
inductive Term extends Term where
  | zero : Term
  | succ : Term → Term
  | natRec : Term → Term → Term → Term
```

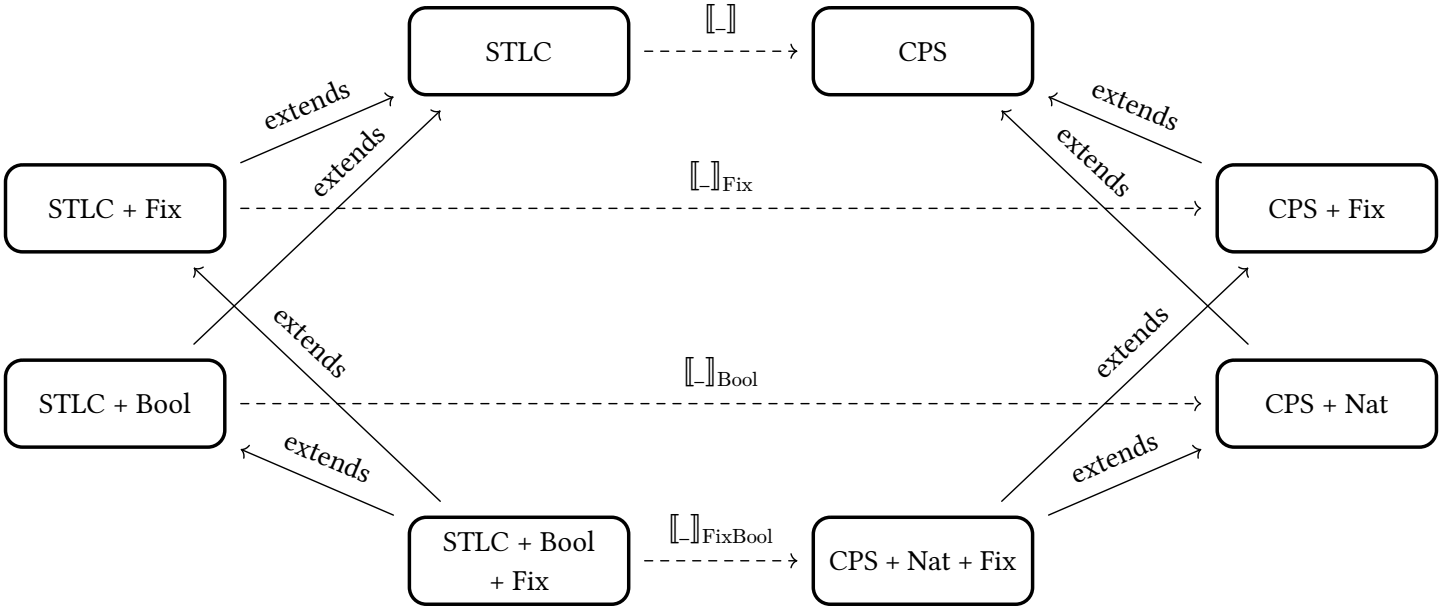
The total formalisation contains 12 inductive types as well as 92 definitions/theorems. Almost all of these were extended correctly. The sole limiting factor in reaching the end of the formalisation relying only on extensions was with regards of the logical relation `LR` used in the original formalisation: their initial formulation was too weak to prove SN on a system extended with natural numbers, and the theorems surrounding it relied heavily on definitional equalities of `LR` that could not be recovered after the partially mapping it to something strong enough for our use-case. As such, the fundamental lemma itself had to be written in a non-extended fashion. A possible solution to circumventing such issues is discussed in the future works.

[1] https://github.com/amarmaduke/lean-stlc

To showcase the capacity to merge separate extensions easily, we produced another extension of STLC containing primitives for booleans, namely a boolean constructor in `Ty`, and term constructors for `true`, `false` and `if-then-else`. We then successfully construct a formalisation of STLC with both natural numbers and booleans by merging the two previous formalisations.

AA: Maybe give some approximation of the numbers of lines saved ?

6. Case study: STLC to CPS translation



To showcase the capabilities of our framework, and provide a point of comparison with another modular system, we reproduce the central case study shown in Pyrosome's (Jamner et al. [2025]) paper. This case study introduces

Explain the challenges of intrinsic verification, how Pyrosome's GAT was circumvented with indexed inductives and quotients.

6.1. Related works

We compare our approach with the recent literature with a special focus on approaches that adapt Data Types à la Carte to proof assistants.

Data Types à la Carte TODO

Coq à la Carte TODO

Pyrosome TODO

Rocqet TODO

AA: TODO talk in details about Rocquet, Datatypes à la Carte, Meta-Theory à la Carte, Pyrosome, Rocq à la Carte

6.2. Future work

- Horizontal extensions
- ETT to ITT translation to circumvent divergences in reduction behaviour between a term and its translation
- Zipper-like structure on expressions to traverse "up" and generalize a goal "after the fact"

References

AWS (2026) Cedar Specification, 2026, https://github.com/cedar-policy/cedar-spec.

Barendregt, H. (1991) An Introduction to Generalized Type Systems, *Journal of Functional Programming*. 1991;1: 125–154., doi:10.1017/S0956796800020025.

Delaware, B., S. Oliveira, B.C. d. and Schrijvers, T. (2013) Meta-theory à la carte, *SIGPLAN Not.* 2013;48(1): 207–218., doi:10.1145/2480359.2429094.

Forster, Y. and Stark, K. (2020) Coq à la carte: a practical approach to modular syntax with binders, In: Blanchette, J. and Hritcu, C. (eds.). *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 20-21, 2020*, ACM 2020: pp. 186–200, doi:10.1145/3372885.3373817.

Goguen, H., McBride, C. and McKinna, J. (2006) Eliminating Dependent Pattern Matching, In: Futatsugi, K., Jouannaud, J.P. and Meseguer, J. (eds.). *Algebra, Meaning, And Computation, Essays Dedicated to Joseph A. Goguen on the Occasion of His 65th Birthday*. Vol 4060. Lecture Notes in Computer Science, Springer 2006: pp. 521–540, doi:10.1007/11780274_27.

Jamner, D., Kammer, G., Nag, R. and Chlipala, A. (2025) Pyrosome: Verified Compilation for Modular Metatheory, *Proc ACM Program Lang*. 2025;9(OOPSLA2)., doi:10.1145/3763052.

Kovács, A. (2020) Elaboration with first-class implicit function types, *Proc ACM Program Lang*. 2020;4(ICFP)., doi:10.1145/3408983.

Leroy, X. (2009) Formal verification of a realistic compiler, *Communications of the ACM*. 2009;52(7): 107–115., http://xavierleroy.org/publi/compcert-CACM.pdf.

Lyons, A., McLeod, K., Klein, G., et al. (2025) seL4/seL4: seL4 14.0.0, December 2025, doi:10.5281/zenodo.17904671.

The mathlib Community (2020) The Lean Mathematical Library, In. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2020, Association for Computing Machinery, New Orleans, LA, USA 2020: pp. 367–381, doi:10.1145/3372885.3373824.

McBride, C. (1999) *Dependently Typed Functional Programs and Their Proofs*, phdthesis, 1999.

Oliveira, B.C. d. S. and Cook, W.R. (2012) Extensibility for the Masses, In: Noble, J. (ed.). *ECOOP 2012 – Object-Oriented Programming*, Springer Berlin Heidelberg, Berlin, Heidelberg 2012: pp. 2–27.

Sozeau, M. and Mangin, C. (2019) Equations Reloaded: High-Level Dependently-Typed Functional Programming and Proving in Coq, *Proc ACM Program Lang*. 2019;3(ICFP)., doi:10.1145/3341690.

Sozeau, M., Anand, A., Boulier, S., et al. (2020) The MetaCoq Project, *Journal of Automated Reasoning*. advance online publication 2020., doi:10.1007/s10817-019-09540-0.

Sozeau, M., Forster, Y., Lennon-Bertrand, M., Nielsen, J., Tabareau, N. and Winterhalter, T. (2025) Correct and Complete Type Checking and Certified Erasure for Coq, in Coq, *Journal of the ACM*. advance online publication January 2025., doi:10.1145/3706056.

Wadler, P. (1998) The expression problem, November 1998, https://homepages.inf.ed.ac.uk/wadler/papers/expression/expression.txt.

7. Appendix A : Formal presentation of extensions

This section presents the technical details of the LeanALaCarte implementation. The main three contributions are 1. A system for partially mapping terms 2. a DSL command for defining an inductive type as an extension to another based on that produces the appropriate partial mappings between the two 3. a DSL for defining definitions/theorems as extensions of others, thus reusing the appropriate informations "after the fact".

AA: TODO section intro AA: For each new thing presented, show the new syntax and how it can be used in practice. Build up an example over the sections using new tools progressively

7.1. Partial mappings

This section makes a partial attempt at justifying the implementation of partial mappings in our system, and why it can be trusted to make well-formed terms.

First, let's define the syntax we'll be using in this toy presentation. This is a basic dependently-typed system equiped with a predicative hierarchy of universes à la Russel, as well as constants and metavariables. For the sake of simplifying the presentation, we will assume constants cannot be unfolded (i.e we will only care about β/η-reduction, not δ-reduction). This, in particular, does not give good justification as to why recursors of inductive types may be extended safely.

$$
\begin{aligned}
\text{Terms} : t, f, A, B ::= {} & x && \text{(variable)} \\
\mid {} & d && \text{(constant)} \\
\mid {} & m && \text{(metavariable)} \\
\mid {} & f\ t && \text{(application)} \\
\mid {} & \lambda(x : A) \mapsto t && \text{(abstraction)} \\
\mid {} & (x : A) \to B && (\Pi\text{-type}) \\
\mid {} & \texttt{Type}_i && \text{(universe)}
\end{aligned}
$$

Listing 1: Syntax of our type theory

The usual typing judgements one would expect apply. These judgement need to carry not only the usual variable context Γ, but also a global constants context Σ to handle the type of constants, similarly to Sozeau et al. [2025], as well as a metavariable context Θ that holds, for each metavariable, both the context and the type of said metavariable, similarly to Kovács [2020]. The idea is that constants may be mapped to some term containing "holes", i.e metavariables, allowing us to make the partial maps.

We define a judgement $\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'$ encompassing the behaviour of the partial mapping in practice. This judgement carries the same contexts found in the aforementionned typing judgements, as well as a mapping context Φ, which maps constants to the bundling of a metavariable context and a term that may contain the metavariables present in the context it's bundled with. The judgement states tracks both the base variable context and a variable context for the partially mapped term, as well as the type of both the original term and the partially mapped one. The partial mapping acts structurally over the usual constructors of our type-theory, and relies on the mapping context to translate constants into their partial mapping. Furthermore, when partially mapping a constant, the metavariable context it carries is weakened/lifted, such that every metavariable lives in some telescope over translated variable context.

$\boxed{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'}$ (In global context Σ, mapping context Φ, metavariable context Θ, term t of type A in context Γ maps to term t' of type A' in context Δ)

AA: TODO fix spacing

$$\frac{}{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash \mathsf{Type}_i \Rightarrow \mathsf{Type}_i : \mathsf{Type}_{i+1} \Rightarrow \mathsf{Type}_{i+1}}$$

$$\frac{(x : A) \in \Gamma \qquad (x : A') \in \Delta}{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash x \Rightarrow x : A \Rightarrow A'} \qquad \frac{((d : A) \mapsto (\Theta, t : A')) \in \Phi}{\Sigma \mid \Phi \mid \Uparrow_\Delta \Theta \mid \Gamma \Rightarrow \Delta \vdash d \Rightarrow t : A \Rightarrow A'}$$

$$\frac{(m : (\Gamma_2, A)) \in \Theta \quad \Gamma_2 \subset \Gamma_1 \quad \Delta_2 \subset \Delta_1 \quad \Sigma \mid \Phi \mid \Theta \mid \Gamma_2 \Rightarrow \Delta_2 \vdash A \Rightarrow A' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i}{\Sigma \mid \Phi \mid \Theta \mid \Gamma_1 \Rightarrow \Delta_1 \vdash m \Rightarrow m : A \Rightarrow A'}$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash f \Rightarrow f' : (x : A) \to B \Rightarrow (x : A') \to B' \\ \Sigma \mid \Phi \mid \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash f\ t \Rightarrow f\ t : B[x := t] \Rightarrow B'[x := t']} \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash A \Rightarrow A' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i \\ \Sigma \mid \Phi \mid \Theta_2 \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A) \vdash B \Rightarrow B' : \mathsf{Type}_j \Rightarrow \mathsf{Type}_j)\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash (x : A) \to B \Rightarrow (x : A') \to B' : \mathsf{Type}_{\max(i,j)} \Rightarrow \mathsf{Type}_{\max(i,j)}} \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash (x : A) \to B \Rightarrow (x : A') \to B' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i \\ \Sigma \mid \Phi \mid \Theta_2 \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A) \vdash t \Rightarrow t' : B \Rightarrow B')\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash (\lambda(x : A) \mapsto t) \Rightarrow (\lambda(x : A') \mapsto t') : (x : A) \to B \Rightarrow (x : A') \to B'} \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing$$

Figure 1: Judgement rules for mappings

The data contained in this judgement is enough to ensure any partially mapped term to be well-typed. Note that this theorem critically relies on the fact that this system only uses

Theorem 1 Given a mapping context Φ, if for each $((d : A) \mapsto (\Theta, t : A')) \in \Phi$:
1. $\Sigma \mid \varepsilon \mid \varepsilon \vdash d : A$
2. $\Sigma \mid \Theta \mid \varepsilon \vdash t : A'$
3. $\Sigma \mid \Phi \mid \Theta \mid \varepsilon \Rightarrow \varepsilon \vdash A \Rightarrow A' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i$ for some i,

Then for all Γ, t, A s.t $\Sigma \mid \Theta \mid \Gamma \vdash t : A$, there exists Δ, t', A' s.t $\Sigma \mid \Theta \mid \Delta \vdash t' : A'$ and $\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'$

Proof: by induction on the typing judgement $\Sigma \mid \Gamma \vdash t : A$. AA: TODO: actually prove this 🥺